REPUBLIC OF THE PHILIPPINES
BICOL UNIVERSITY
POLANGUI
COMPUTER STUDIES DEPARTMENT
Polangui, Albay

ISO 9001: 2015
SOCOTEC SCP000722Q

BAGONG PILIPINAS

IT 112: Web Systems and Technologies
BSIT- 2A 2nd Semester
S.Y. 2025-2026

Final Project – Phase 5: Displaying and Managing Data on the Frontend
Combat Coders: The End of Bugs Reign

Submitted by:

TEAM PLASMA
Sophia Mae Alodia J. Balmaceda
Joshua Cedric B. Velitario
Dwayne Lester O. Diaz
Kian Carlo O. Matutina
Hafsah A. Tuzon

Submitted to:
Dr. Guillermo V. Red, Jr, DIT

April 2026

Introduction

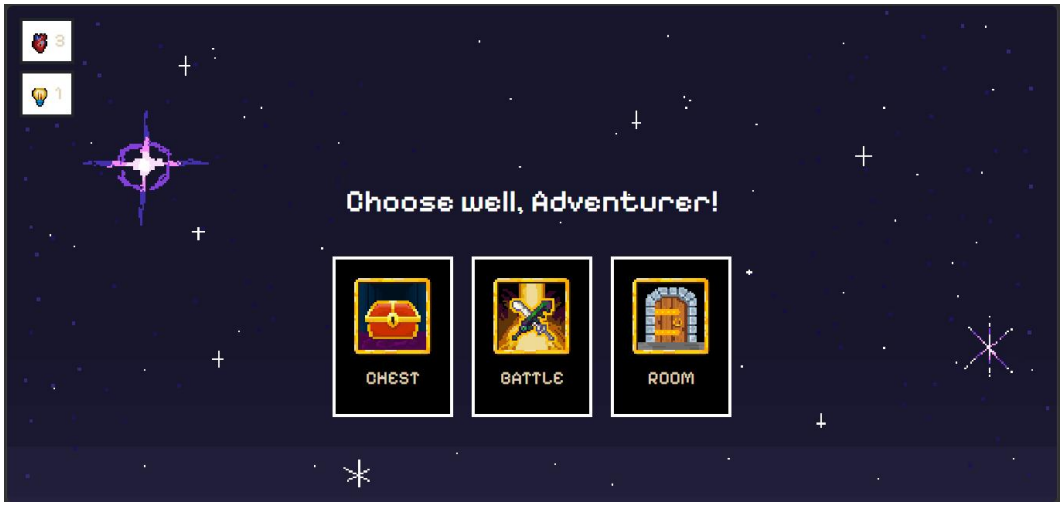
This phase of the project focuses on integrating the frontend and backend by enabling dynamic data retrieval and display. Building on the previously developed system structure, the objective of this lab is to connect the user interface to backend API endpoints and present real-time data using HTML and JavaScript. Through the use of fetch() or similar methods, data is retrieved in JSON format and rendered dynamically into user-friendly components such as dashboards, tables, and cards.

Additionally, this phase emphasizes the creation of reusable interface templates, basic data control features like search or pagination, and proper team collaboration through GitHub. By completing these tasks, the team enhances both the functionality and usability of the system, ensuring that the application can efficiently handle and present live data while maintaining a clean and responsive interface.

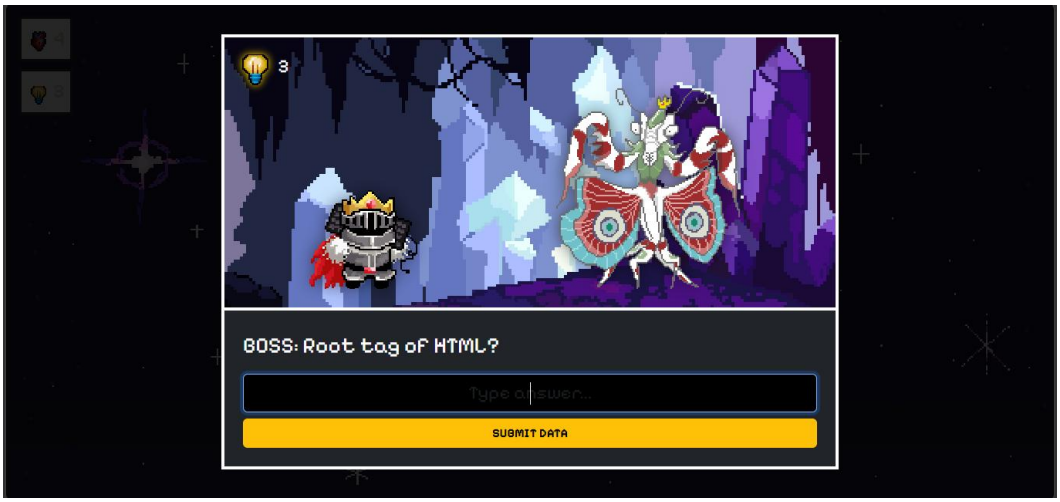
I. Screenshot of at Least 2 Display Pages

The following screenshots present display pages of the system, showcasing the user interface and navigation flow of the application.

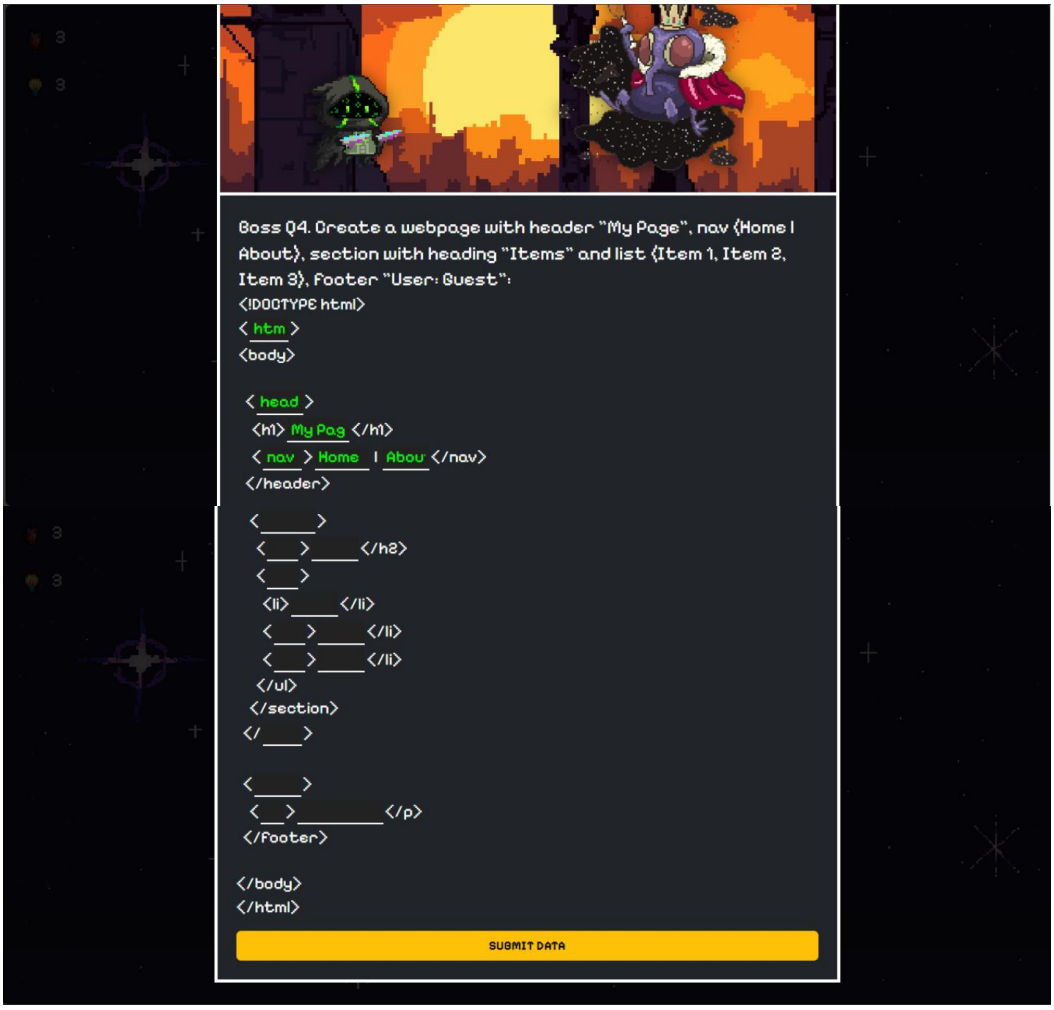
(Home)



(Paths)



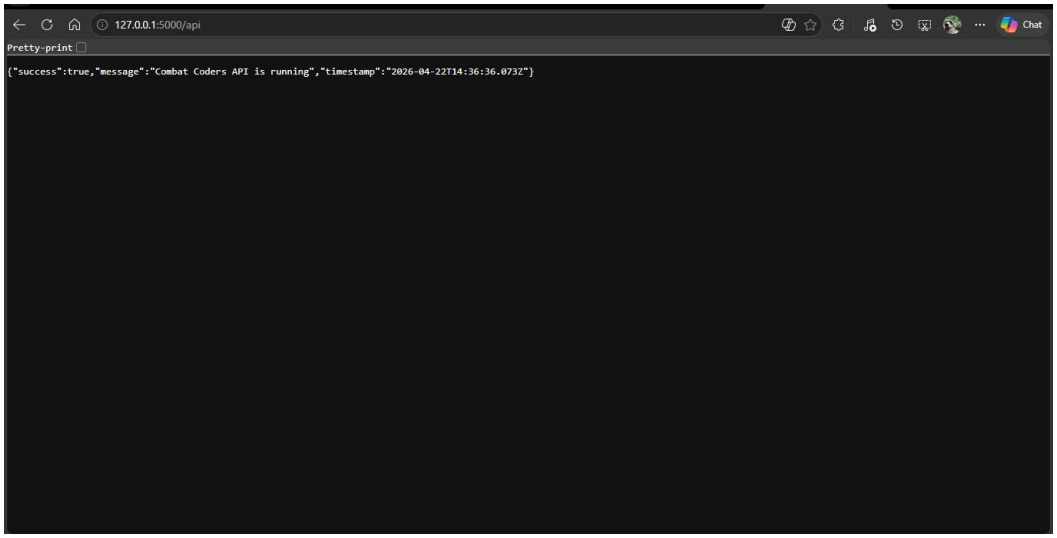
(Battle)

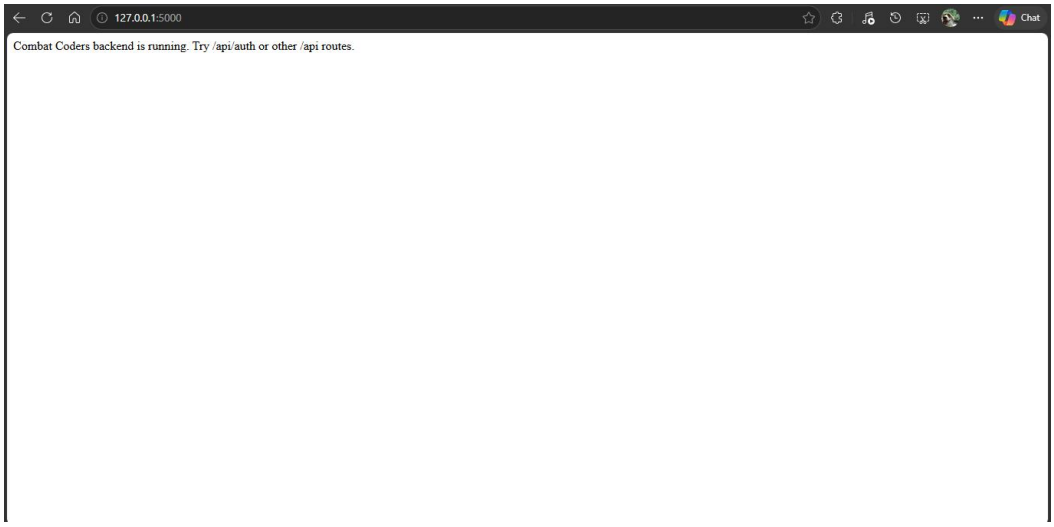


(Boss Battle)

II. Sample Backend API Response (Console or Postman Screenshot)

The following screenshots show sample responses from the backend API, tested through the browser and API tools.

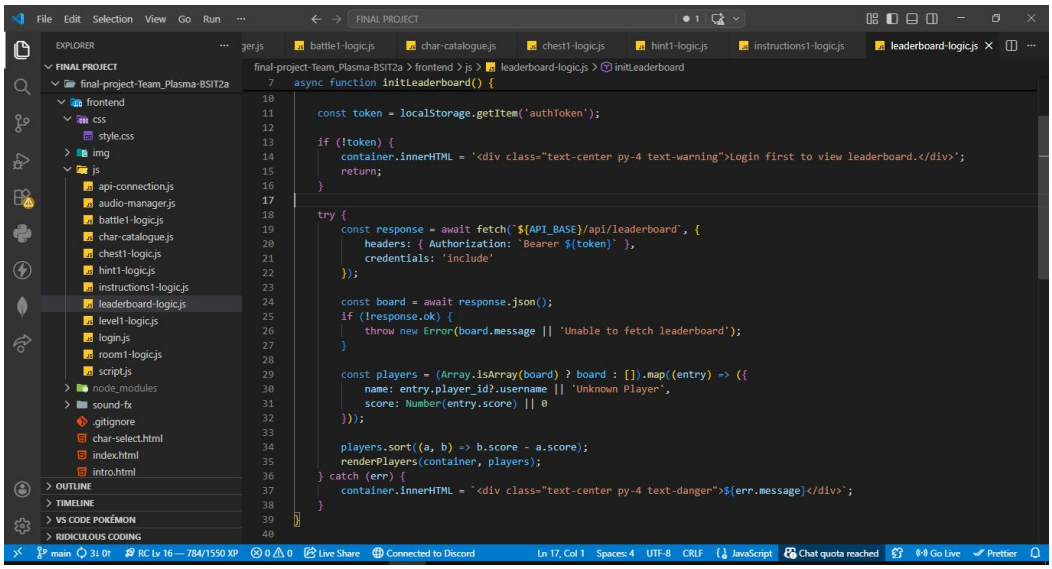




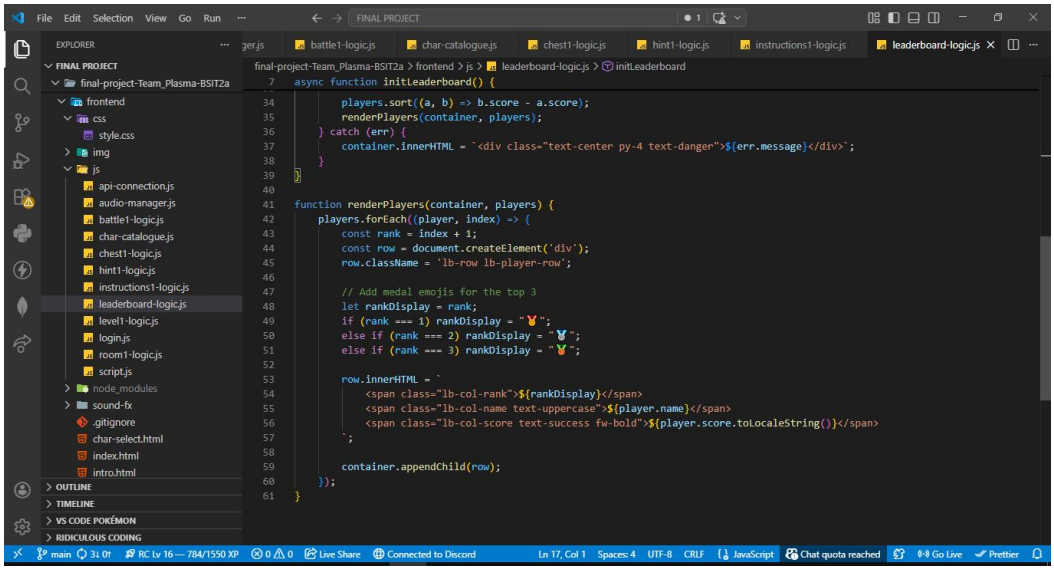
III. Annotated JavaScript Snippet Used For Rendering Data

The following JavaScript snippets demonstrate how data is retrieved from the backend API and dynamically rendered onto the web page.

The code shown in the images retrieves leaderboard data from the backend, processes and sorts it, and dynamically displays it on the web page. It verifies user authentication, formats player data, assigns rankings, and uses DOM manipulation to render the results, with error handling for failed requests or unauthorized access.



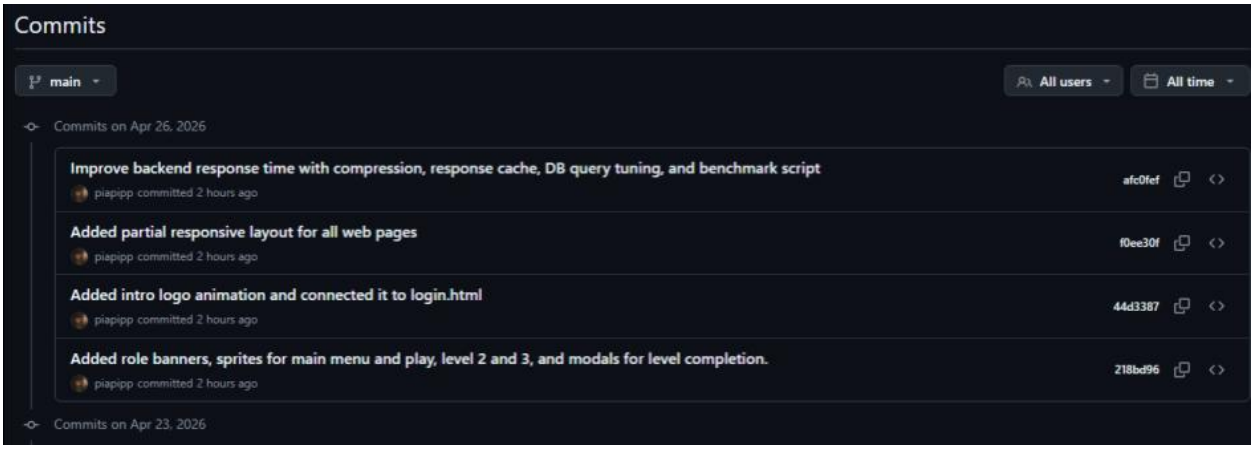

```
7   async function initLeaderboard() {
10
11      const token = localStorage.getItem('authToken');
12
13      if (!token) {
14          container.innerHTML = '<div class="text-center py-4 text-warning">Login first to view leaderboard.</div>';
15          return;
16      }
17
18      try {
19          const response = await fetch(`${API_BASE}/api/leaderboard`, {
20              headers: { Authorization: `Bearer ${token}` },
21              credentials: 'include'
22          });
23
24          const board = await response.json();
25          if (!response.ok) {
26              throw new Error(board.message || 'Unable to fetch leaderboard');
27          }
28
29          const players = (Array.isArray(board) ? board : []).map((entry) => ({
30              name: entry.player_id?.username || 'Unknown Player',
31              score: Number(entry.score) || 0
32          }));
33
34          players.sort((a, b) => b.score - a.score);
35          renderPlayers(container, players);
36      } catch (err) {
37          container.innerHTML = `<div class="text-center py-4 text-danger">${err.message}`;
38      }
39  }
40
```

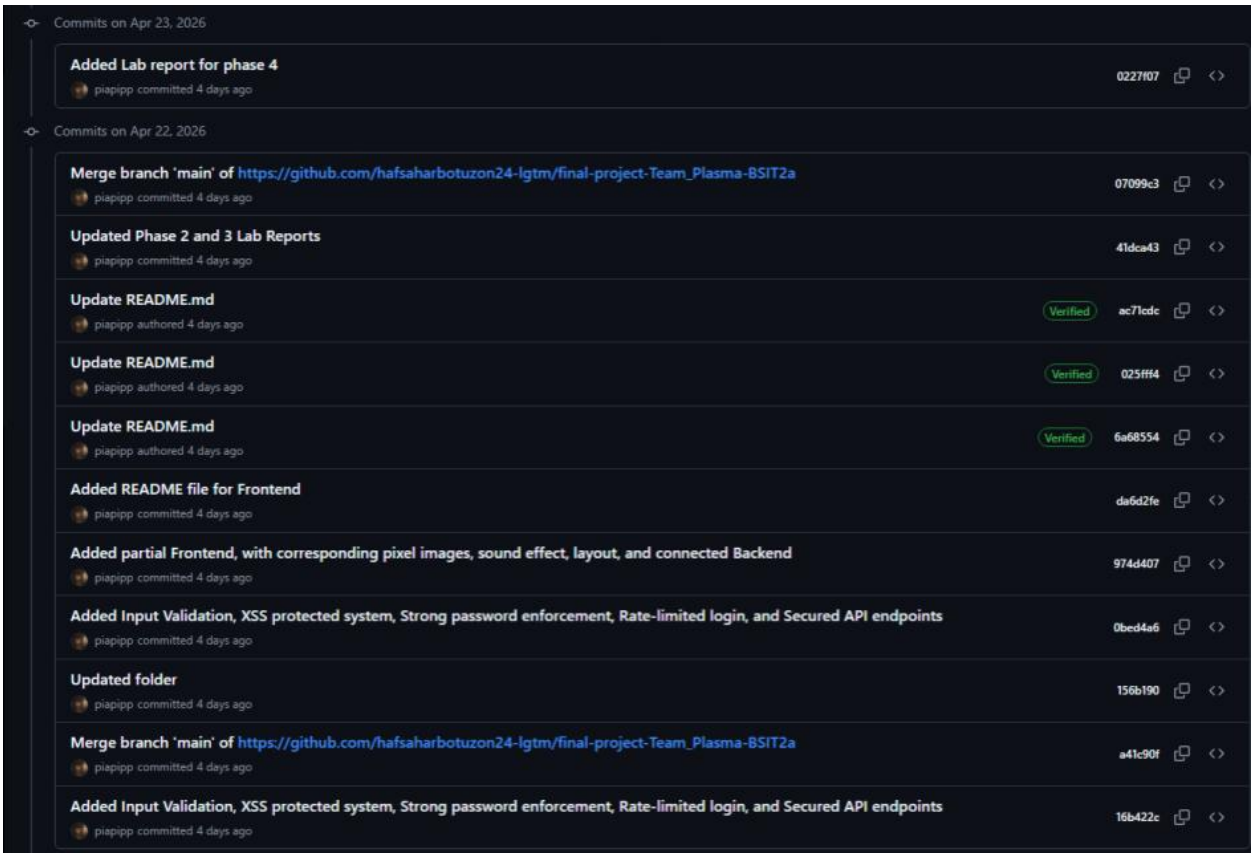



```
7   async function initLeaderboard() {
34          players.sort((a, b) => b.score - a.score);
35          renderPlayers(container, players);
36      } catch (err) {
37          container.innerHTML = `<div class="text-center py-4 text-danger">${err.message}`;
38      }
39  }
40
41  function renderPlayers(container, players) {
42      players.forEach((player, index) => {
43          const rank = index + 1;
44          const row = document.createElement('div');
45          row.className = 'lb-row lb-player-row';
46
47          // Add medal emojis for the top 3
48          let rankDisplay = rank;
49          if (rank === 1) rankDisplay = "🥇";
50          else if (rank === 2) rankDisplay = "🥈";
51          else if (rank === 3) rankDisplay = "🥉";
52
53          row.innerHTML = `
54              <span class="lb-col-rank">${rankDisplay}</span>
55              <span class="lb-col-name text-uppercase">${player.name}</span>
56              <span class="lb-col-score text-success fw-bold">${player.score.toLocaleString()}</span>
57          `;
58
59          container.appendChild(row);
60      });
61  }
```

(Codes from leaderboard-logic.js)

IV. **GitHub Commit Logs**





V. Short Reflections From Each Member

Sophia Mae Alodia J. Balmaceda: *Backend Developer and Project Manager*

Worked on making sure the GET endpoints return the correct data format. Improved the system's security. Also helped in building parts of the frontend.

Dwayne Lester O. Diaz: *Frontend Developer*

Focused on creating dynamic HTML interfaces. Used JSON responses from the backend to display data. Helped connect the frontend with backend outputs.

Joshua Cedric B. Velitario: *Database Manager*

Completed the questions, answers, and hints for each battle. Helped organize game content. Assisted in finishing the lab report.

Kian Carlo O. Matutina: *Documentation Officer*

Finalized on making the questions, answers, and hints for each boss in the game. Made the player avatars/icons to be selected in-game. And also, managed the laboratory report and finalized it.

Hafsah A. Tuzon: *GitHub Manager*

Finished the backgrounds, icons, and visual elements for each game scenario. Helped improve the overall design of the game. Assisted in completing the lab report for the week.